Constellate Architecture

System design for a multi-modal coaching agent built with Gemini 3

Overview

Constellate is a multi-agent system that maintains coaching continuity across sessions through structured memory, composes dynamic UI interactions, and generates personalized behavioral interventions using Gemini 3's multi-modal capabilities.

System Architecture

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        LangGraph Dev Server (:2024)
│
│   ┌──────────────────────────────────────────────────────────────────────┐
│   │                        Coach Agent (DeepAgent)                        │
│   │  Model: gemini-3-pro-preview                                          │
│   │  Tools: fan_out (A2UI)                                                │
│   │  Subagents: intervention_composer                                     │
│   │  Memory: /user/coach/AGENTS.md, /user/profile/context.md              │
│   │                                                                       │
│   │   ┌──────────────────────────────────────────────────────────────┐   │
│   │   │            Intervention Composer (Subagent)                   │
│   │   │  Model: gemini-3-flash-preview                                │
│   │   │  Tool: compose_experiential_intervention                      │
│   │   │          └─> gemini-3-pro-preview (Image API)                 │
│   │   │          └─> A2UI interrupt (image + caption + feedback)      │
│   │   └──────────────────────────────────────────────────────────────┘   │
│   └──────────────────────────────────────────────────────────────────────┘
│                    │
│                    │
│   ┌──────────────────────────────────────────────────────────────────────┐
│   │                        CompositeBackend                               │
│   │  /user/      → StoreBackend (persistent cross-session)                │
│   │  /scratch/   → StateBackend (temporary within-session)                │
│   └──────────────────────────────────────────────────────────────────────┘
│
│
│   ┌──────────────────────────────────────────────────────────────────────┐
│   │  ModelRegistry                  │  PromptRegistry                     │
│   │  coach → gemini-3-pro           │  coach_system.j2                    │
│   │  intervention_composer          │  intervention_composer_system       │
│   │    → gemini-3-flash             │    .j2                              │
│   └──────────────────────────────────────────────────────────────────────┘
│
│  SummarizationMiddleware (85% context triggers compression)
└──────────────────────────────┬──────────────────────────────────────────────
                               │ useStream + interrupt
┌──────────────────────────────┴──────────────────────────────────────────────
│                         Next.js Frontend
│  ChatContainer → MessageList + InputBar
```

```
│  FanOutPanel (slide-in: half / three-quarter / full) → A2UIRenderer
│  BottomTabBar → Coach / Map / Journal
└──────────────────────────────────────────────────────────────────────
```

Core Components

1. Coach Agent (DeepAgent)

Role: Main conversational interface, maintains coaching relationship across sessions

Configuration:

```
agent = create_deep_agent(
    name="coach",
    model=ModelRegistry.get("coach"),  # gemini-3-pro-preview
    system_prompt=PromptRegistry.get("coach_system"),
    backend=composite_backend,
    memory=["/user/coach/AGENTS.md", "/user/profile/context.md"],
    tools=[fan_out],
    subagents=[intervention_composer],
)
```

Key Capabilities:

- **Cross-session memory:** MemoryMiddleware auto-loads persistent memory files into system prompt
- **File system tools:** Built-in tools (ls, read_file, write_file, edit_file, glob, grep) for behavior ledger manipulation
- **Dynamic UI composition:** `fan_out` tool for A2UI interactions
- **Subagent delegation:** Delegates experiential intervention creation to specialist subagent

Context Management:

- `SummarizationMiddleware` compresses conversation at 85% context utilization
- Full history stored to `/conversation_history/{thread_id}.md`
- Coach edits `/user/coach/AGENTS.md` to update long-term memory

2. Intervention Composer (Subagent)

Role: Specialist agent for assembling experiential interventions using Gemini 3

Tool: `compose_experiential_intervention`

- Constructs image generation prompts following theoretical frameworks
- Calls Gemini 3 Image API (`gemini-3-pro-preview`)
- Returns A2UI payload (image + caption + feedback select component)

Intervention Types:

Type	Theory	Purpose
`future_self`	Hershfield (Future Self-Continuity)	Build identity consistency across time

`scene_rehearsal`	MCII/WOOP (Oettingen)	Mental rehearsal for high-risk situations
`metaphor_mirror`	Lakoff & Johnson (Conceptual Metaphor)	Preserve and expand user's metaphors
`reframe_contrast`	CBT (Cognitive Reframing)	Dual-perspective visualization

Ethical Constraints (prompts/intervention_composer_system.j2 lines 55-66):

- Forbidden content: idealized bodies, body comparison, weight/shape focus
- Required framing: behavioral capability, situational mastery, values
- User control: consent required, dismissal/feedback options

Caching: Module-level cache prevents duplicate API calls on interrupt resume

3. A2UI System (Agent-to-UI Protocol)

Concept: Composable UI primitives for dynamic agent-driven interactions

7 Component Primitives:

```
# Display components
text(content: str)
image(src: str, alt: str)

# Input components
slider(name: str, label: str, min: int, max: int, step: int, value: int
text_input(name: str, label: str, placeholder: str, value: str)
number_input(name: str, label: str, unit: str, value: float)
select(name: str, label: str, options: List[{label, value}], value: str
multi_select(name: str, label: str, options: List[{label, value}], valu
```

Unified Interrupt Protocol:

```
{
  "type": "a2ui",
  "components": [{"kind": "text", "content": "..."}, ...],
  "layout": "half" | "three-quarter" | "full"
}
```

Resume Protocol:

```
{
  "action": "submit" | "reject" | "skip",
  "data": {"component_name": "value", ...}
}
```

Implementation:

- Backend: `src/constellate/a2ui.py` (Pydantic discriminated union on `kind`)
- Tool: `src/constellate/tools/fan_out.py`
- Frontend: `frontend/src/components/fanout/A2UIRenderer.tsx`

Usage Pattern:

1. Coach calls `fan_out(components, layout)`
2. Payload validated → `interrupt()` → frontend FanOutPanel
3. User interacts → `resume(A2UIResponse)`
4. Tool returns result → Coach continues

4. Behavioral Ledger (Virtual File System)

Data Model: Event-sourced behavior history

Directory Structure:

```
/user/
├── profile/
│   └── context.md              # User identity, goals, preferences
├── ledger/
│   └── {YYYY-MM-DD}/           # Daily directory
│       └── {HHMMSS}_{type}.json  # Event file with timestamp
├── derived/
│   ├── weekly_trend.json       # Aggregated patterns
│   └── pattern_log.md          # Pattern recognition history
└── coach/
    └── AGENTS.md               # Coach persistent memory
```

Event Schema (coach-defined, not enforced by backend):

- All events: `ts`, `type`, `evidence` (user quotes), `tags`
- Event types: `meal`, `exercise`, `weigh_in`, `state_checkin`, `sleep`, `goal`, `reflection`, `craving`, `app_action`, custom...

Writing Strategy:

- Async event recording (user silence ≥5 min or session end)
- One event per file
- Evidence field required (references user's words)

Backend Routing:

Path	Backend	Persistence
`/user/`	StoreBackend	Cross-session persistent
`/scratch/`	StateBackend	Within-session temporary

Demo: InMemoryStore (clears on restart) **Production:** Supabase-backed custom backend (JSONB columns + path indexing)

5. Configuration Registries

ModelRegistry (`src/constellate/config/models.py`):

- Central LLM configuration with lazy loading
- TOML-based config with environment variable interpolation
- Example: `config/models.toml`

```
[models.coach]
model = "google_genai:gemini-3-pro-preview"
```

```
    api_key = "${GEMINI_API_KEY}"

    [models.intervention_composer]
    model = "google_genai:gemini-3-flash-preview"
    api_key = "${GEMINI_API_KEY}"
```

PromptRegistry (`src/constellate/config/prompts.py`):

- Jinja2 template engine for system prompts
- Templates in `prompts/` directory
- StrictUndefined mode for fail-fast on missing variables

Data Flow

Standard Conversation Flow

```
User message
  → LangGraph runtime
    → Coach Agent
      → Process with system prompt + memory
      → (Optional) Call fan_out for UI interaction
        → interrupt() → Frontend FanOutPanel
          → User input → resume()
      → (Optional) Write to behavior ledger
      → Response message
    → Frontend MessageList
```

Experiential Intervention Flow

```
Coach detects intervention opportunity
  → Delegate to intervention_composer subagent
    → Subagent constructs intervention prompt
      → Call compose_experiential_intervention tool
        → Generate image via Gemini 3 Pro Image API
        → Assemble A2UI payload (image + caption + select)
        → interrupt() propagates: tool → subagent → coach → client
          → Frontend renders full-screen A2UI panel
            → User: accept / dismiss / mark unhelpful
              → resume() with feedback
        → Tool returns result
      → Subagent completes
    → Coach continues with intervention outcome
```

Technology Choices

Why Gemini 3?

- **gemini-3-pro-preview:**
  - Main Coach Agent (complex reasoning, coaching continuity, nuanced conversations)
  - Image generation (mixed text/image output, ethical constraints adherence,

bounding boxes)

- **gemini-3-flash-preview:**
  - Intervention Composer (fast prompt assembly)
  - Summarization (context compression)
- Cost-effective balance between capability and scale

Why LangGraph + DeepAgent?

- Virtual file system abstraction (agent-native data manipulation)
- Built-in memory middleware (auto-loading persistent context)
- Interrupt/resume protocol (HITL interventions)
- Subagent composition (separation of coaching vs. intervention assembly)

Why A2UI over fixed forms?

- Coaching conversations are inherently unpredictable
- Same primitives serve multiple purposes (data collection, micro-interventions, experiential delivery)
- Single interrupt protocol simplifies frontend rendering
- Composability enables unlimited interaction patterns from 7 base components

Why event-sourced ledger over normalized database?

- Preserves full behavior context (evidence quotes, confidence levels, tags)
- Schema evolution without migrations (coach defines structure)
- Natural alignment with EMA research methodology
- Supports both structured queries (patterns) and full-text search (evidence)

Deployment

Docker (Recommended)

The project includes Docker Compose configuration for one-command setup:

```
cp .env.example .env   # Add GEMINI_API_KEY
docker compose up --build
# Frontend: http://localhost:3000
# Backend:  http://localhost:2024
```

- `Dockerfile` — Python 3.12 + uv backend (LangGraph API server)
- `frontend/Dockerfile` — Next.js multi-stage build (standalone output)
- `docker-compose.yml` — Service orchestration with health checks

Scalability Considerations

Current (Demo):

- Single-user InMemoryStore
- Docker Compose or local LangGraph dev server
- No authentication

Production Path:

- Custom Supabase backend implementing BackendProtocol
- File paths → JSONB columns with GIN indexing
- Row-Level Security for multi-user isolation
- Vercel deployment for frontend
- LangGraph Cloud for agent runtime

No agent code changes required — backend is pluggable interface

Security & Ethics

Content Safety

- Ethical constraints enforced in intervention_composer system prompt
- No user-facing content generation without guardrails
- Intervention feedback loop for safety refinement

Data Privacy

- Behavior ledger is user-owned (can export/delete)
- No third-party analytics on sensitive data
- Coach memory is user-specific, never cross-contaminated

Transparency

- Generated content explicitly labeled as AI-generated exploration
- User consent required before experiential interventions
- Coaching decisions visible in conversation flow (no hidden manipulation)

Testing Strategy

Unit Tests:

- A2UI component validation (`tests/test_a2ui.py`)
- Event schema validation (agent-driven, not enforced)
- Model/Prompt registry (`tests/test_model_registry.py`)

Integration Tests:

- Coach agent conversation flows
- Subagent delegation + interrupt/resume cycles
- Fan-out tool + A2UI rendering

Manual Testing:

- LangGraph Studio for conversation debugging
- Step-through execution in Graph mode
- State inspection at each node

Future Enhancements

Multi-modal input:

- Voice journaling (Gemini 3 audio transcription)
- Wearable data integration (sleep, activity)

Advanced interventions:

- Video-based scenario rehearsal
- Interactive decision tree simulations

Cross-domain extension:

- Sleep coaching (same architecture, different domain knowledge)
- Exercise adherence (same S-PDCA methodology)

Collaborative coaching:

- Peer accountability groups
- Coach-assisted goal setting workshops